**in-memory database provider doesn't simulate a database as such. Instead, it stores objects in memory and executes LINQ queries against them directly.**

**NOTE** In this section I describe how to use the SQLite provider as an in-memory database, as it's more full-featured than the in-memory provider. For details on using the in-memory provider, see Microsoft's "EF Core In-Memory Database Provider" documentation: http://mng.bz/hdIq.

To use the SQLite provider in memory, add the Microsoft.EntityFrameworkCore.Sqlite package to your test project's .csproj file. This adds the `UseSqlite()` extension method, which you'll use to configure the database provider for your unit tests.

Listing 36.13 shows how you could use the in-memory SQLite provider to test the `GetRecipe()` method of `RecipeService`. Start by creating a `SqliteConnection` object and using the `"DataSource=:memory:"` connection string. This tells the provider to store the database in memory and then open the connection. This is typically faster than using a file-based connection-string and means you can easily run multiple tests in parallel, as there's no shared database.

**WARNING** The SQlite in-memory database is destroyed when the connection is closed. If you don't open the connection yourself, EF Core closes the connection to the in-memory database when you dispose of the `DbContext`. If you want to share an in-memory database between `DbContexts`, you must explicitly open the connection yourself.

Next, pass the `SqliteConnection` instance into the `DbContextOptionsBuilder<>` and call `UseSqlite()`. This configures the resulting `DbContextOptions<>` object with the